Nevermind, this is ALL Phoenix specific

BIOS

- 24k BOOT ROM (embedded in bitstream) located from 0x0000 to 0x5FFF. Actually initialized RAM so it will be writable. Note that written values will persist between reset
- ColecoVision BIOS is no longer visible or replaceable
- Read from port **>55** to switch the BIOS: 24k disappears and 8k BIOS appears at 0x0000 as ROM
- There is no way back via software - hardware reset only.

Cartridge

- Detected as a normal cartridge by looking for a standard cart header. Upon reset the real cart port will be active.
- During BIOS (bootload), the physical cartridge port can be read at address >8000 .. >FFFF
- Disabled when a mapper is selected for cartridge emulation from RAM, i.e. when port >55 two LS-bits are other than "00"

SD Card

- The SD card can be read/written at port **>57**
- Accesses transfer 8-bits at a time, full duplex.
- The active-low chip-enable control will be via port >56 bit >01
- An SD card can be detected by reading port >56 and checking bit >80 (1=card inserted). The chip-enable status will also be returned in bit >01.

RAM Access

- 512KiB of External SRAM is available on the system. Access is organized as 16 banks of 32K each.
- The selected 32K bank will be at the cartridge address space from >8000 .. >FFFF when a banking scheme is selected (port >55 two LS-bits are other than "00").
- Bank 0 is reserved for the SGM 32K RAM and is only visible as RAM from >0000 to >7FFF when the SGM is enabled. The remaining 15 pages (1..15) are used for MegaCart or other bank-switching systems.
- To allow the loader access to the pages during game loading, the bank scheme should be other than "00" and the upper memory operation set to "00".
- Only after the cartridge is loaded, should the actual upper memory operation be selected to enable the required banking scheme.

I/O Ports

```
Port  CPU Data
--------------
 >50  WWWWWWWW (W) WSG / SGM-PSG Address Write
 >51  WWWWWWWW (W) WSG / SGM-PSG Data Write
 >52  RRRRRRRR (R) WSG / SGM-PSG Data Read
 >53  xxxxx__W (W) 0 = SGM disable, 1 = SGM enable
      xxxxx_0_ (W) 0 = SGM Sound AY-8910, 1 = Alt (unimplemented)
      xxxxxW__ (W) 0 = PS/2 joystick, 1 = PS/2 Keyboard port 0x80

 >54  xxxxWWWW (W) 32K RAM bank select. Starts at 1, writes of 0 are
                   ignored. (Operation may differ when a banking
                   scheme is enabled!)
      xxxxRRRR (R) current bank

 >55  xxxxxxxx (R) Loader disable, PoR re-enables loader
      xx__xxWW (W) Banking scheme
            00     None, physical cartridge, other bits ignored
            01     MegaCart: bank is address access >FFC0+bank
            10     SGM: Bank is data written to >FFFC,>FFFD,>FFFE
            11     Atari: >FF80, >FF90, >FFA0, >FFB0 ???
      xxWWxx__ (W) Select the upper memory operation:
        00         32K Upper Internal RAM, ignores bank scheme
        01         32K Expansion ROM, uses bank scheme
        10         32K Upper Expansion RAM, uses bank scheme
        11         32K Cartridge ROM, ignores bank scheme

 >56  xxxxxx_W (W) SD-card CE_n (AKA SS_n), 0=enable
      xxxxxxW_ (W) SD-card speed, 1=400KHz, 0=12MHz
      RxxxxxRR (R) SD-card card-detect, bit >80 1=card inserted
 >57  WWWWWWWW (W) SD-card Data Write
      RRRRRRRR (R) SD-card Data Read

 >58  00001000 (R) Machine ID, Phoenix = 8

 >59  WWWWWWWW (W) Cartridge page mask for banking
 >5A  xxxxxxxx (R/W) These two are additional mapping registers for
 >5B  xxxxxxxx (R/W) the SGM mode, along with >54. Not normally
                     accessed directly, but exposing them anyway.

 >7F  xxxxxxWx (W) 0 = 8K SGM RAM, 1 = 8K CV BIOS
```

```
>80  xxxxxxxx (W) Any write selects keypads (0x80-0x9F)
     RRRRRRRR (R) if enabled, read gets last PS/2 scancode


>BE  WWWWWWWW (W) VDP write data (VDP regs repeat over 0xA0-0xBF)
     RRRRRRRR (R) VDP read data
>BF  WWWWWWWW (W) VDP write address
     RRRRRRRR (R) VDP read status

>C0  xxxxxxxx (W) Any write selects joysticks (0xC0-0xDF)

>FC  RRRRRRRR (R) Read joystick or keypad 1 (repeats >E0-FF)
>FF  RRRRRRRR (R) Read joystick or keypad 1
>FF  WWWWWWWW (W) Write SN sound chip
```

Mapper comments from Mike… this is how I've implemented it in Classic99 and the loader at the moment:

Cartridge mapper assumptions:

We have two registers: **0x54** and **0x55**. These control what appears in the cartridge memory space from **0x8000-0xFFFF**.

0x54 is simply a *page* register. There are 16 pages (32k each) in the system, although page 0 is reserved for use by the SGM and can not be used for cartridge emulation. If page 0 is selected, the page will not be changed, i.e. it will remain at the current bank (page). The bank (page) register can never have the value "0000", and is set to "0001" at power-on and reset.

0x55 contains the *mapper* information. There are two sets of bits that have meaning. The bits with the mask **0x03** define the banking *scheme*, and the bits with mask **0x30** define *how to apply* it.

The first and most important bit pair is 0x03:

00 - this means that the physical cartridge port is active. *All other settings are ignored.*
01 - this means that emulation is active, and Megacart banking will be employed (see below).
10 - this means that emulation is active, and SGM banking will be employed *(not implemented)*.
11 - this means that emulation is active, and Atari banking will be employed *(not implemented)*.

Thus for these bits today, **only 00 and 01 have meaning**.

The other bitmask is 0x30:

00 - the cartridge space contains 32k of RAM, with the 32k page selected by port 0x54. Used by the loader.
01 - the cartridge space contains 32k of ROM, with a banking scheme selected by the 0x03 bits active.
10 - the cartridge space contains 32k of RAM, with a banking scheme selected by the 0x03 bits active.
11 - the cartridge space contains 32k of ROM, with what ever bank is selected by the bank register.

We should probably ask ourselves whether to lock ports 0x54 and 0x55 after the loader exits. Otherwise badly written software could manipulate the ports and break the emulation.

It might also be nice to have a 'Phoenix' mapper that lets us leave the ports unlocked, so that titles can use the 512k of RAM in the style they prefer (maybe instead of 'SGM' or 'Atari' banking... we could use a different cartridge header to detect those titles).

Matthew's comments to Mike's comments about the banking (aka paging):

Agree, we may need to lock ports 54 and 55, but I would like to leave them unlocked until it becomes a problem. The SGM is susceptible to the same problem for the ports it adds.

The Bank (Page) Select Register (BSR) set by port 54 can actually be modified in two ways: 1. Directly via port 54, and 2. Indirectly via the selected banking scheme. I will have more details when I actually implement the MegaCart banking. ;-)

Thus, bits 0x30 of port 55 (the UMR or Upper Memory-Mode Register) can be considered as an enable for banking, and a write enable:

00 - write enable active (memory is RAM), indirect banking ignored
11 - write enable inactive (memory is ROM), indirect banking ignored
In both cases, only port 54 can select the bank.

10 - write enable active (RAM), indirect banking active
01 - write enable inactive (ROM), indirect banking active

The 11 and 01 selections (ROM) are the typical settings for existing hardware. The RAM variations were modeled after the ADAM's upper 32K memory use and will probably never be utilized, but they do exist if anyone wants to take advantage of them.

Megacart Banking

Megacart EPROMs are broken up into 16k pages, and are banked by accessing the last 64 addresses of the cartridge space (0xFFC0 - 0xFFFF). Accessing 0xFFFF maps the top 16k of the EPROM (or page 0),

0xFFFE maps the second-to-last 16k (or page 1), 0xFFFD maps the third-to-last 16k (or page 2), etc. (Naturally the page is masked by the number of pages you actually have). Megacarts are limited to a maximum size of 1MB.

The cartridge space itself is divided into two 16k segments. From 0x8000 to 0xBFFF is a fixed segment which always displays the last 16k page on the Megacart (page 0). from 0xC000 to 0xFFFF is a banked segment that displays the currently selected page.

Megacarts do not have a fixed startup page for the banked memory.

The current plan is to load Megacarts at the TOP of memory, so the fixed page will appear appear in Phoenix memory at 512k-16k, and the rest down from there. For now we're just acknowledging that the last 32k won't be available, and we are restricted to Megacarts that are 512k-32k and don't use those "last" two pages. This actually should cover pretty much everything for now.

We need to have a way to communicate a page mask (or size of the actual ROM). In Classic99 I calculate a page number and mask that. Using that assumption, valid sizes are:

512k - 32 banks - mask 0x1F
256k - 16 banks - mask 0x0F
128k - 8 banks - mask 0x07
64k - 4 banks - mask 0x03
32k - 2 banks - mask 0x01 - *(not likely to exist in the wild, but it's free to implement)*

I vote for a new register (0x59) if it's feasible, rather than bitslicing another one.

Math is something like (the NOT to set the MSB bits high):

Latched_Bits AND Mask_Value OR (NOT Mask_Value)

Maximum value is 0x1F

Matthew Notes for MegaCart.

I need to keep reminding myself that the mask is needed to make the Phoenix act like real hardware. Without the mask, software would have

to be written to always select their pages using the correct page from the top of the selection range.

It seems that MegaCart detection (vs other paging schemes) might be reliably detected by looking for the 55AA or AA55 signature at the offset (FileSize - 16384), since the fixed bank is always the last 16K in a file.

Loading can be done by taking (512K - FileSize) and using that as the starting offset in the 512K RAM, and then just copying linearly from there.

Menu operation so far

Following does not cover the megacart design changes yet.

At reset, the menu assumes that the physical port is selected. If no cart is detected, it then waits for an SD to be inserted.

On SD detection, it sets port 0x55 to 0x01 (megacart banking mode, upper internal RAM). This assumes 512k RAM mapped with 0x54 selecting the page, but ignoring the megacart part (only selected to turn off the physical cartridge port).

Port 0x54 is set to 14 - second to last page. This was to keep the data out of the way of both normal and megacart cartridge loads, even if we manage to get 512k carts going later. (I can load the first and last page for verification without overwriting the directory info).

The SD is then read and all data is stored in this mapped 32k of space.

Once a game is selected, we change port 0x54 to ‘1’ for the first page of memory (and, the assumed only page for a 32k cart). We load the first 512 bytes and check for a cartridge header.

If the 32k cartridge header is present, we load the rest of the cartridge. Then port 0x55 is changed to 0x31 (32k ROM with no paging, megcart paging mode - this assumes that page 1 will be mapped (or at least that whatever I left mapped is left alone)), and the title is started.

If the 32k cartridge header was not present, then I seek ahead to 16k from the end of the image, and read another 512 bytes. If there is a cartridge header here, then I know I have a Megacart.

If it's a Megacart, we calculate the starting page and then load it into the memory from there (so that it ends at the top of 512k RAM). We then write the megacart size mask to register 0x59 (32k is 0x1, 64k is 0x03, 128k is 0x07, 256k is 0x0f, 512k is 0x1f). Next we set the banking scheme to 0x11 (Banking scheme megacart, upper banked ROM). The last thing set before starting the title is to reset the bank select at 0x54 to 1, but this is not expected to do anything.

When a title starts, first I turn off the screen and interrupts in VDP register 1. My code next resets the F18A and then initializes the ColecoVision RAM at 0x6000 with the same memory patterns that the ColecoVision BIOS would, and also prepares the VDP. (This init is skipped if the cartridge header is set to skip it).

Finally, it copies a short trampoline into ColecoVision RAM at 0x6000 (the only deliberate deviation, but it seems safe) and jumps to it. This trampoline reads port 0x55 to disable the loader and swap in the Coleco BIOS, sets the stack pointer to the Coleco BIOS value, loads the cartridge boot vector and jumps to it.

SGM Banking

Although Opcode said there were two possible approaches, I believe we've only seen one. At any rate, this is what I implemented and it was enough to run both Gradius and Penguin Adventure in Classic99:

There's pretty much no overlap with Megacart, so we need to absolutely select one or the other.

SGM banking works in 8k pages, and there are three addresses that are used to control the paging. Each one is conceivably an 8-bit value, which supports 2MB, but since we only have 512k we can only support up to 6 bits anyway (with the same 'gotcha' that Megacarts have with 512k, we lose 32k of that space to RAM, but 256k and below are fine.)

Only writes matter on this device (reads won't drive the data bus and you'll end up selecting a random bank).

Cartridge address 0x8000 always has the first 8k of the ROM.
Cartridge address 0xA000 is banked, selected by writing to 0xFFFC
Cartridge address 0xC000 is banked, selected by writing to 0xFFFD
Cartridge address 0xE000 is banked, selected by writing to 0xFFFE

Address 0xFFFF is I/O used for communication with a (serial?) EEPROM device. I don't know the part number. I also don't know how reads and writes are differentiated. It's possible that this concept was not implemented, as neither Penguin nor Gradius seem to have an EEPROM. There's probably more to it than that, for now we don't have enough information for that part.

According to KevTris, Activision style cart:

The "Activision" EEPROM games include Boxxle, Arcomage, Jewel Panic, and The Black Onyx.

RFF80 - returns data in bank 2 or 3 depending on if SDA is low or high, respectively (only at this address, see below)
WFF90 - select bank 1 @ C000
WFFA0 - select bank 2 @ C000
WFFB0 - select bank 3 @ C000
WFFC0 - clear SCL
WFFD0 - set SCL
WFFE0 - clear SDA

WFFF0 - set SDA

ROMs appear to return 00 and 01 at FF80 (BF80, FF80 in the 64K ROM itself) depending on the state of SDA. I suspect, going by the contents, that the last 128 bytes of the cart is fixed to these two ranges (bank 2 or bank 3's last 128 bytes) depending on the state of SDA at all times, but didn't check. The last 128 bytes of the three selectable banks are all FF's except the first byte at FF80 of bank 2 and 3 as specified.

Bank 0 is fixed to 8000-BFFF.